

ЗАДАНИЕ 1

Исходное отношение представляет собой плоскую таблицу с атрибутами: AnimalID, Animal, Species, Age, Owner, DateOfVisit, Manipulation, в которой каждая строка описывает одну манипуляцию, выполненную над конкретным животным в определённую дату. В такой структуре данные о животном и его владельце многократно повторяются, что приводит к избыточности.

На первом этапе необходимо сформировать функциональные зависимости, отражающие семантику предметной области. Идентификатор животного однозначно определяет его кличку, вид и возраст, а также владельца, поэтому выполняются зависимости вида $\text{AnimalID} \rightarrow \text{Animal, Species, Age, Owner}$. Одновременно предполагается, что составной ключ ($\text{AnimalID, DateOfVisit, Manipulation}$) однозначно определяет каждую запись об отдельной процедуре. Наличие зависимостей от части составного ключа указывает на нарушение требований второй нормальной формы, так как описательные атрибуты животного и владельца зависят только от AnimalID, а не от полного ключа строки.

Для устранения частичных зависимостей исходное отношение декомпозируется. Выделяется сущность «Животное» с отношением Animal ($\text{animal_id, animal_name, species, age, owner_id}$), где первичный ключ animal_id определяет все неключевые атрибуты. Отношения, описывающие визиты, переносятся в таблицу Visit ($\text{visit_id, animal_id, date_of_visit}$), в которой используется идентификатор визита, а дата визита полностью функционально зависит от visit_id . Далее устраняются транзитивные зависимости: сведения о владельце выносятся в отдельное отношение Owner ($\text{owner_id, owner_name}$), а в таблице Animal остаётся только внешний ключ owner_id . Аналогичным образом процедуры выносятся в справочник Manipulation ($\text{manipulation_id, manipulation_name}$), что позволяет хранить описание каждой процедуры единожды.

Так как один визит может включать несколько процедур, между визитами и манипуляциями формируется связь «многие-ко-многим», реализуемая отношением VisitManipulation ($\text{visit_id, manipulation_id}$), где составной первичный ключ ($\text{visit_id, manipulation_id}$) обеспечивает уникальность каждой операции в рамках визита. В результате итоговая схема базы данных включает пять отношений: Owner ($\text{owner_id, owner_name}$), Animal ($\text{animal_id, animal_name, species, age, owner_id}$), Visit ($\text{visit_id, animal_id, date_of_visit}$), Manipulation ($\text{manipulation_id, manipulation_name}$), VisitManipulation ($\text{visit_id, manipulation_id}$). В каждой из этих таблиц все неключевые

атрибуты полностью функционально зависят от первичного ключа и не зависят транзитивно от других неключевых атрибутов, что удовлетворяет формальному определению третьей нормальной формы и позволяет хранить данные с минимальной избыточностью и высокой логической целостностью.

Финальная схема

Отношение: Owner (owner_id, owner_name); **ФЗ:**

- owner_id → owner_name

Отношение: Animal (animal_id, animal_name, species, age, owner_id); **ФЗ:**

- animal_id → animal_name,
- animal_id → species
- animal_id → owner_id
- animal_id → age

Отношение: Visit (visit_id, animal_id, date_of_visit); **ФЗ:**

- visit_id → animal_id
- visit_id → date_of_visit

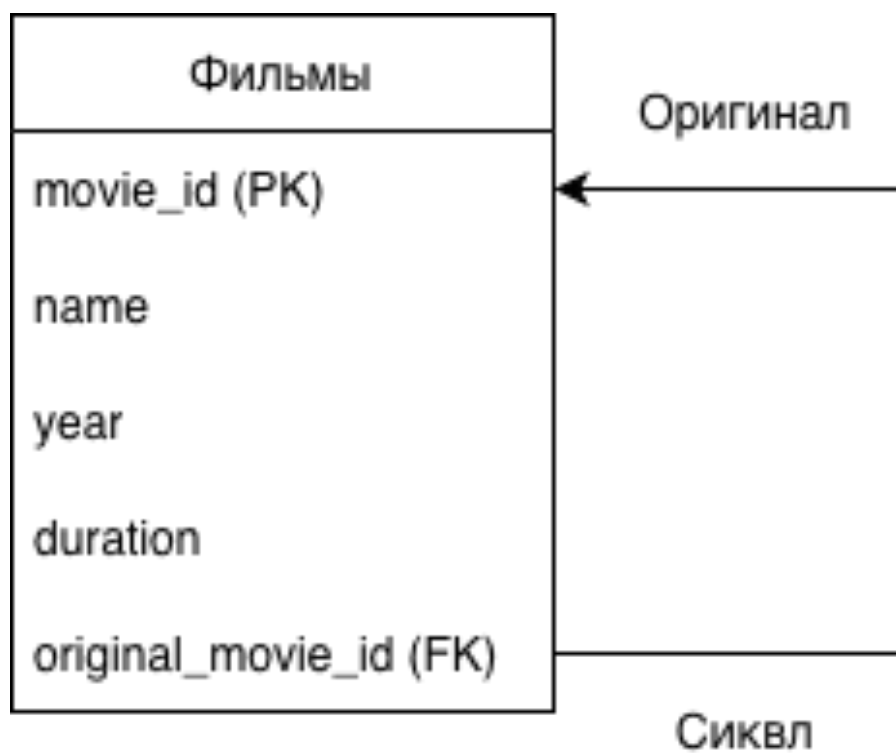
Отношение: Manipulation (manipulation_id, manipulation_name); **ФЗ:**

- manipulation_id → manipulation_name

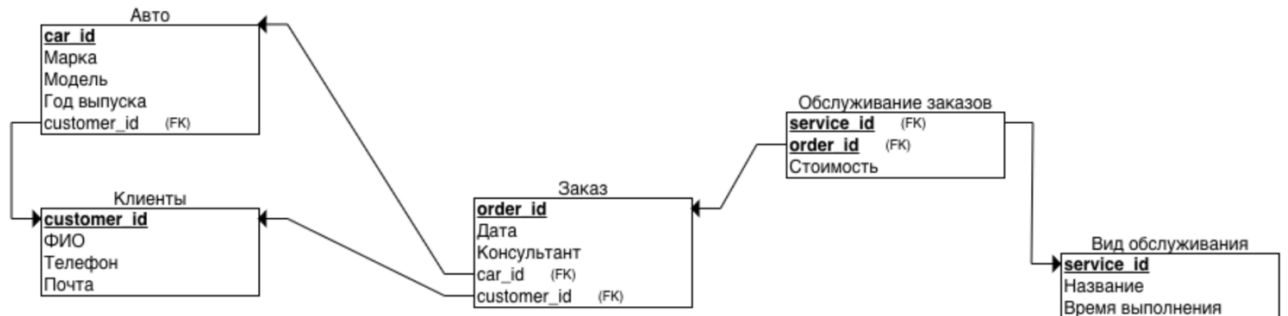
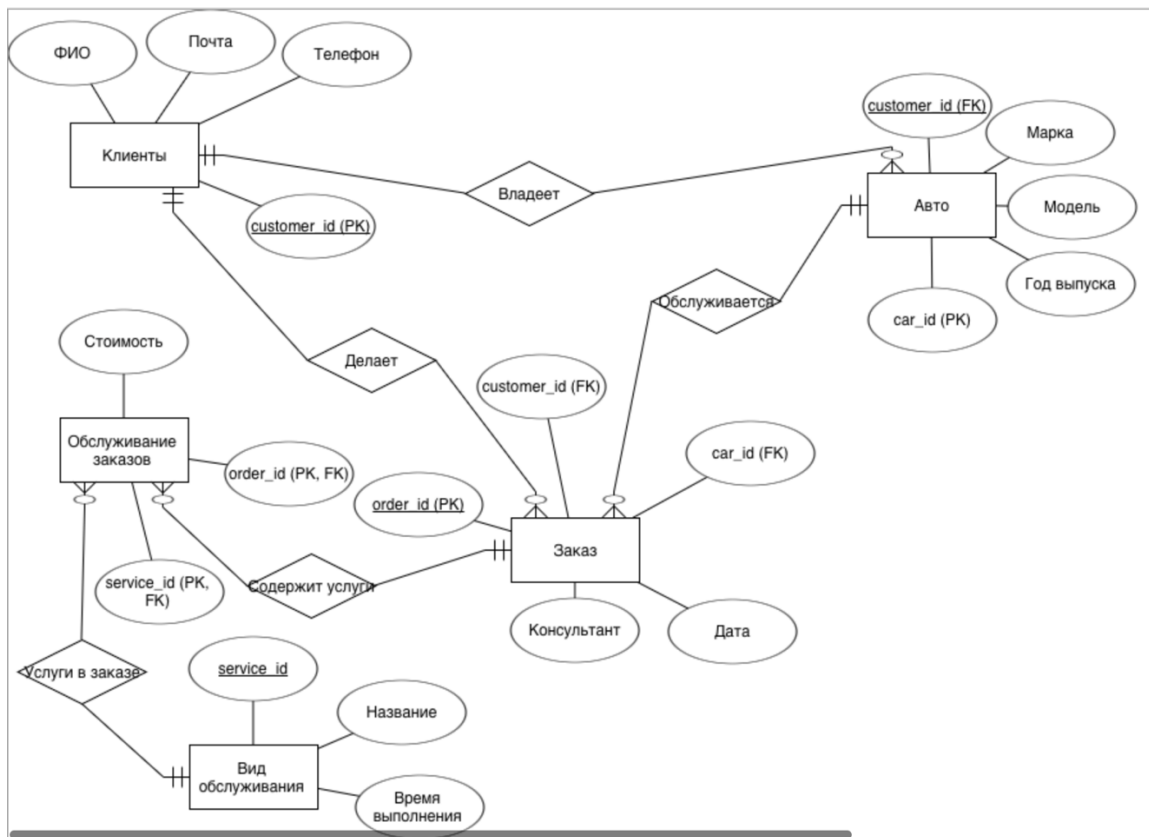
Отношение: VisitManipulation (visit_id, manipulation_id); **ФЗ:**

- от составного первичного ключа других атрибутов нет

ЗАДАНИЕ 2



ЗАДАНИЕ 3



```

CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    email VARCHAR(100)
);

CREATE TABLE Cars (
    car_id INT PRIMARY KEY,
    mark VARCHAR(50) NOT NULL,
    model VARCHAR(50) NOT NULL,
    prod_year SMALLINT,
    customer_id INT NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

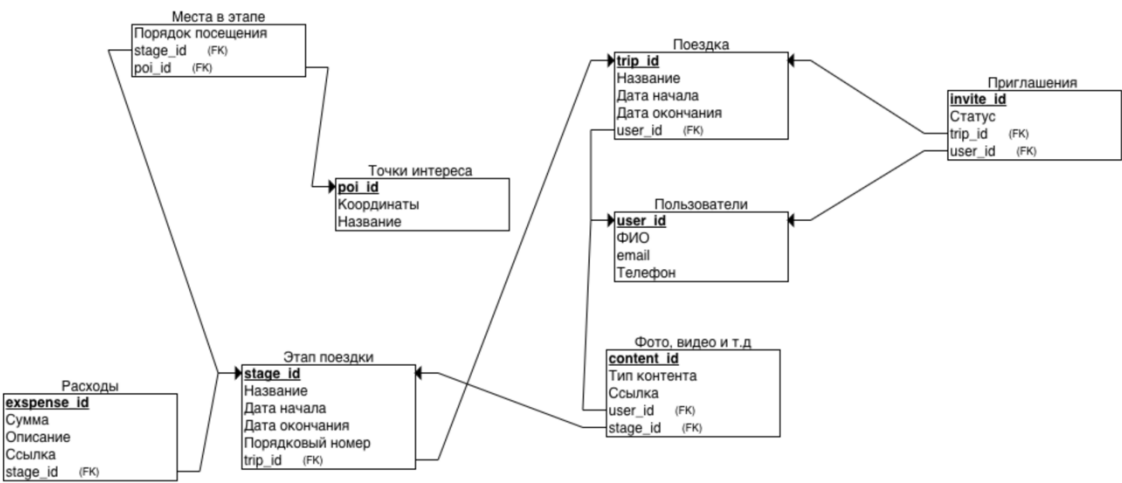
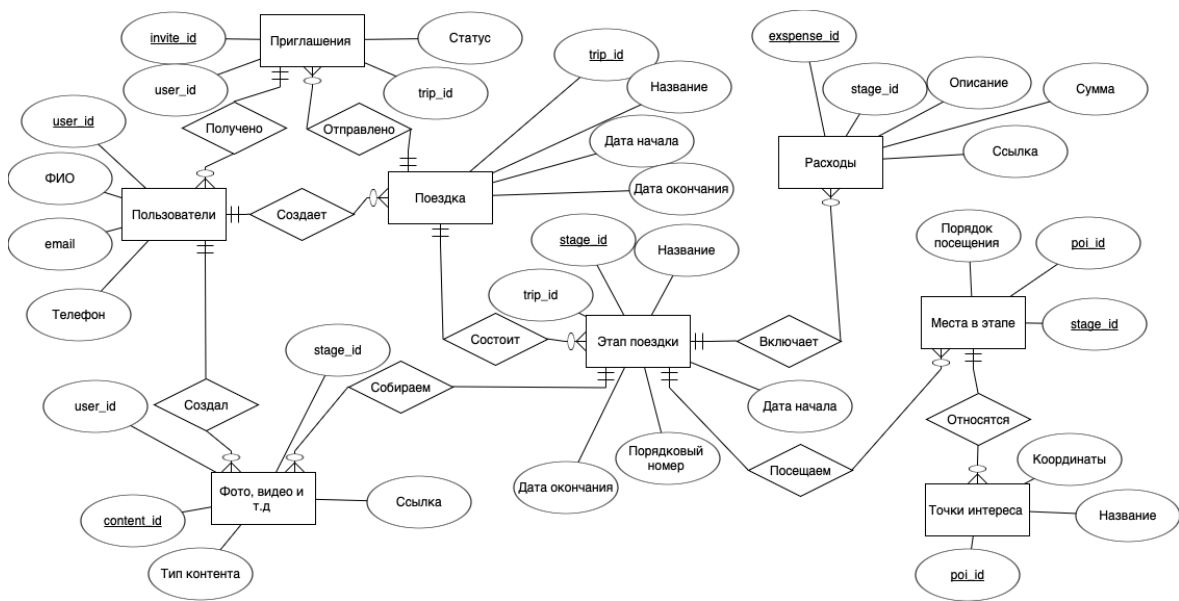
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    order_date DATE NOT NULL,
    consultant VARCHAR(100) NOT NULL,
    car_id INT NOT NULL,
    customer_id INT NOT NULL,
    FOREIGN KEY (car_id) REFERENCES Cars(car_id),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

CREATE TABLE ServiceTypes (
    service_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    planned_hours DECIMAL(4,2) NOT NULL
);

CREATE TABLE OrderServices (
    service_id INT NOT NULL,
    order_id INT NOT NULL,
    cost DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (service_id, order_id),
    FOREIGN KEY (service_id) REFERENCES ServiceTypes(service_id),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);

```

ЗАДАНИЕ 4



```
CREATE TABLE Users (  
    user_id INT PRIMARY KEY,  
    full_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    phone VARCHAR(30)  
);  
  
CREATE TABLE Trips (  
    trip_id INT PRIMARY KEY,  
    title VARCHAR(200) NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE,  
    user_id INT NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);  
  
CREATE TABLE TripStages (  
    stage_id INT PRIMARY KEY,  
    trip_id INT NOT NULL,  
    title VARCHAR(200) NOT NULL,  
    start_date DATE,  
    end_date DATE,  
    stage_number INT,  
    FOREIGN KEY (trip_id) REFERENCES Trips(trip_id)  
);  
  
CREATE TABLE PointsOfInterest (  
    poi_id INT PRIMARY KEY,  
    name VARCHAR(200) NOT NULL,  
    coordinates VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE PlacesInStage (  
    stage_id INT NOT NULL,  
    poi_id INT NOT NULL,  
    visit_order INT,  
    PRIMARY KEY (stage_id, poi_id),  
    FOREIGN KEY (stage_id) REFERENCES TripStages(stage_id),  
    FOREIGN KEY (poi_id) REFERENCES PointsOfInterest(poi_id)  
);  
  
CREATE TABLE Expenses (  
    expense_id INT PRIMARY KEY,  
    stage_id INT NOT NULL,  
    amount DECIMAL(10,2) NOT NULL,  
    description VARCHAR(255),  
    link_url VARCHAR(255),  
    FOREIGN KEY (stage_id) REFERENCES TripStages(stage_id)  
);  
  
CREATE TABLE Contents (  
    content_id INT PRIMARY KEY,  
    content_type VARCHAR(50) NOT NULL,  
    link_url VARCHAR(255),  
    user_id INT NOT NULL,  
    stage_id INT NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id),  
    FOREIGN KEY (stage_id) REFERENCES TripStages(stage_id)  
);  
  
CREATE TABLE Invitations (  
    invite_id INT PRIMARY KEY,  
    status VARCHAR(20) NOT NULL,  
    trip_id INT NOT NULL,  
    user_id INT NOT NULL,  
    FOREIGN KEY (trip_id) REFERENCES Trips(trip_id),  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```